TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

CẨM NANG LUYỆN NÓI & PHẢN BIỆN CÁ NHÂN
HỒ ĐOÀN NGUYÊN KHANG (1923040112)

Vai trò: API Engineer – Phụ trách ASP.NET Core Web API, Swagger & Database | Thứ tự thuyết trình: Người số 3 (Trình bày Kỹ thuật Lõi Backend) | Thời lượng mục tiêu: 4 phút 15 giây (Khóa cứng thời gian)
Dự án: Hệ thống Quản lý Hoạt động Hội nhập – VISIONARY

📌 **CHIẾN THUẬT DÀNH CHO HỒ KHANG:** Hồ Khang chịu trách nhiệm trình bày phần 'nặng' nhất về kiến trúc Web API: Swagger UI, chu trình 6 chặng của một yêu cầu, phân biệt bản chất 401 vs 403, JWT Token Lifecycle và thiết kế DTO/Database. Giảng viên chấm điểm rất kỹ việc chỉ đúng tên file, class, method và dòng code kiểm tra quyền trên server.

I. PHÂN BỔ SLIDE VÀ THỜI GIAN (4 PHÚT 15 GIÂY)

Slide	Tiêu đề Slide	Nội dung trọng tâm	Thời lượng
Slide 8	Swagger & Hợp Đồng API	Bảng hợp đồng API chuẩn, các endpoint công khai / cần quyền Admin, Swagger UI	1 phút 00 giây
Slide 9	Xuyên Suốt Một Yêu Cầu (6 Chặng)	Chu trình 6 chặng từ Click -> Handler -> HTTP -> Server/DB -> JSON -> UI Render	1 phút 15 giây
Slide 10	Xác Thực & Phân Quyền (JWT, 401/403)	Vòng đời JWT Token, phân biệt rạch ròi 401 vs 403, server không tin client	1 phút 00 giây
Slide 11	JSON, Database & Quyết Định Kỹ Thuật	Thiết kế DTO, phân trang PagedResponse, kết nối PostgreSQL Supabase & AsNoTracking	1 phút 00 giây

II. LỜI THUYẾT TRÌNH TỪNG CÂU (HỌC THUỘC LÒNG & NÓI TỰ NHIÊN)

1. Khi ở Slide 8 (Swagger & Hợp Đồng API) – 1 phút 00 giây:

"Kính chào Thầy và các bạn, em là Hồ Đoàn Nguyên Khang. Tiếp nối phần demo của Trí Khang, em xin đi sâu vào trái tim của toàn bộ hệ thống: ASP.NET Core 8 Web API và Hợp đồng API.

Nhóm áp dụng triệt để nguyên tắc 'API First': Mọi ứng dụng client muốn kết nối đều phải đọc tài liệu Swagger trước để nắm rõ cấu trúc JSON và phương thức HTTP. Trên slide là ảnh chụp giao diện Swagger

thực tế đang chạy tại địa chỉ cloud của nhóm.

Bảng hợp đồng thể hiện rõ: Endpoint GET /api/events là mở công khai để khách và sinh viên cùng xem danh sách. Endpoint POST /api/events/{id}/register yêu cầu xác thực người dùng thường. Và đặc biệt endpoint POST /api/events tạo sự kiện mới bắt buộc phải có quyền Admin hoặc Ban Chủ nhiệm (Committee). Mọi tên trường JSON trả về đều tuân thủ nghiêm ngặt chuẩn camelCase."

2. Khi ở Slide 9 (Xuyên Suốt Một Yêu Cầu - 6 Chặng) – 1 phút 15 giây:

"Thưa Thầy, để chứng minh tính nhất quán xuyên suốt, nhóm phân tích chi tiết đường đi của một yêu cầu tải danh sách sự kiện GET /api/events qua đúng 6 chặng:
- Chặng 1: Người dùng nhấn nút tải hoặc mở màn hình sự kiện.
- Chặng 2: Hàm xử lý giao diện được kích hoạt (Web là useEffect, Android là onResume, WinForms là Form_Load).
- Chặng 3: Client gửi HTTP GET /api/events kèm tham số phân trang. Trên Android chạy qua Background Thread của Retrofit, WinForms chạy qua Task của HttpClient.
- Chặng 4: Web API nhận request tại EventsController.GetEvents(), gọi EventService truy vấn PostgreSQL qua Entity Framework Core với AsNoTracking để tối ưu hiệu năng.
- Chặng 5: Server tuần tự hóa dữ liệu thành JSON chuẩn ApiResponse chứa PagedResponse và trả về HTTP 200.
- Chặng 6: Từng client nhận JSON: Web map vào State, WinForms nạp vào DataGridView, Android nạp vào RecyclerView. Cả ba nền tảng đều hiển thị chính xác cùng một tập dữ liệu."

3. Khi ở Slide 10 (Xác Thực & Phân Quyền) – 1 phút 00 giây:

"Về bảo mật, nhóm triển khai cơ chế JWT Bearer Token theo chuẩn RFC 7519. Khi đăng nhập thành công qua POST /api/auth/login, server phát một chuỗi JWT gồm 3 phần: Header, Payload chứa Claim danh tính, và Signature được ký bằng khóa bí mật HMAC-SHA256.

Đặc biệt, nhóm phân biệt rạch ròi giữa hai mã lỗi bảo mật:
- Mã 401 Unauthorized: Xảy ra khi client KHÔNG gửi token, hoặc token đã HẾT HẠN (hết 4 giờ). Server từ chối ngay ở Middleware xác thực.
- Mã 403 Forbidden: Xảy ra khi client CÓ token hợp lệ (đã biết là sinh viên Nguyễn Văn A), nhưng tài khoản này KHÔNG ĐỦ QUYỀN để gọi API quản trị (chỉ dành cho Admin/BCN).

Nguyên tắc cốt lõi: Server tuyệt đối không tin client. Danh tính sinh viên được lấy trực tiếp từ Claim NameIdentifier trong Token, không cho phép client tự gửi mã tài khoản lên trong body để mạo danh."

4. Khi ở Slide 11 (JSON, Database & Quyết Định Kỹ Thuật) – 1 phút 00 giây:

"Về tầng dữ liệu, hệ thống lưu trữ trên PostgreSQL tại Supabase. Nhóm kiên quyết sử dụng lớp DTO (Data Transfer Object) thay vì trả trực tiếp Entity của cơ sở dữ liệu. Lý do là để che giấu các trường nhạy cảm như PasswordHash, ngăn chặn lỗi vòng lặp tuần tự hóa JSON, và tối ưu dung lượng gói tin qua mạng.

Tất cả danh sách đều được bọc trong vỏ phân trang PagedResponse gồm items, pageNumber, pageSize, totalCount và totalPages, ngăn ngừa tình trạng quá tải bộ nhớ khi dữ liệu phình to.

Sau đây, em xin chuyển lời sang bạn Võ Ngọc Minh Đạt để trình bày về ứng dụng Windows Forms, tổng kết các khác biệt kỹ thuật và phân tích sự cố thực tế của dự án. Xin mời anh Đạt!"

III. MÃ NGUỒN CẦN MỞ VÀ 3 ĐIỂM NÓNG GIẢNG VIÊN DỄ TRỎ VÀO

Tệp mã nguồn Khang phụ trách mở khi giảng viên kiểm tra Backend:

📁 File Controller: backend/src/Visionary.Api/Controllers/EventsController.cs

📁 File Program.cs: backend/src/Visionary.Api/Program.cs

📁 File Service: backend/src/Visionary.Core/Services/EventService.cs

📘 Mã nguồn minh chứng: Kiểm tra quyền và trích xuất danh tính trong EventsController.cs

```
// EventsController.cs
[HttpPost("{id}/register")]
[Authorize] // Bắt buộc phải có JWT Token (không có -> 401)
public async Task<IActionResult> Register(int id) {
    // Đọc danh tính trực tiếp từ Token, client không thể giả mạo
    var userIdStr = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (string.IsNullOrEmpty(userIdStr)) return Unauthorized();

    var res = await _eventService.RegisterAsync(id, int.Parse(userIdStr));
    return StatusCode(res.StatusCode, res);
}

[HttpPost]
[Authorize(Roles = "Admin,Committee")] // Phân quyền Server-side (không đủ quyền -> 403)
public async Task<IActionResult> Create([FromBody] CreateEventRequest req) { ... }
```

👉 Điểm 1: Thuộc tính [Authorize(Roles = "Admin,Committee")]:

Giảng viên hỏi: Thuộc tính này hoạt động thế nào? Nếu một user có vai trò 'Student' gọi vào thì nhận mã gì?
-> Khang trả lời: 'Dạ thưa Thầy, Middleware Authorization của ASP.NET Core sẽ giải mã Token, đọc ClaimTypes.Role. Nếu user chưa đăng nhập, trả về 401 Unauthorized. Nếu user đã đăng nhập với vai trò Student (không thuộc Admin hay Committee), server trả về ngay mã 403 Forbidden mà không thực thi nội dung hàm.'

👉 Điểm 2: Dòng trích xuất User.FindFirstValue(ClaimTypes.NameIdentifier):

Giảng viên hỏi: Tại sao không truyền studentId trong body hay query string cho tiện?
-> Khang trả lời: 'Dạ thưa Thầy, nếu nhận studentId từ body thì bất kỳ ai cũng có thể sửa request bằng Postman để đăng ký hoặc hủy sự kiện thay cho người khác. Trích xuất từ Claim trong JWT đảm bảo 100% người thực hiện chính là người đã đăng nhập thành công vì chữ ký số không thể bị làm giả.'

👉 Điểm 3: Truy vấn phân trang AsNoTracking() trong EventService.cs:

Giảng viên hỏi: AsNoTracking() dùng để làm gì? Xóa đi có chạy được không?
-> Khang trả lời: 'Dạ thưa Thầy, AsNoTracking báo cho Entity Framework Core không theo dõi thay đổi (Change Tracking) của các thực thể đọc từ DB. Đối với API đọc dữ liệu (Read-only), việc này giúp tiết kiệm bộ nhớ RAM và tăng tốc độ truy vấn lên tới 30-40%. Xóa đi code vẫn chạy nhưng hiệu năng sẽ giảm sút.'

IV. BỘ CÂU HỎI PHẢN BIỆN CHUYÊN SÂU WEB API & DATABASE

❓ Câu 1: JWT có phải là một chuỗi mã hóa không? Bất kỳ ai cũng có thể đọc được nội dung của JWT đúng không?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, JWT KHÔNG PHẢI là chuỗi mã hóa bí mật (Encryption) mà là chuỗi được KÝ SỐ (Digital Signature) và mã hóa Base64URL. Bất kỳ ai có token đều có thể decode phần Payload để đọc thông tin Claim. Tuy nhiên, không ai có thể sửa đổi Payload vì Server sẽ kiểm tra tính toàn vẹn bằng chữ ký số với Secret Key.

❓ Câu 2: Dòng nào bật CORS trong Program.cs? Nếu xóa đi thì nền tảng nào bị lỗi?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, CORS được đăng ký tại builder.Services.AddCors(...) và kích hoạt bằng app.UseCors("AllowAll"). Nếu xóa đi, CHỈ CÓ ứng dụng Web trên trình duyệt bị lỗi do chính sách Same-Origin Policy của trình duyệt chặn lại. Hai ứng dụng Windows Forms và Android vẫn hoạt động hoàn toàn bình thường.

❓ Câu 3: Nếu client gửi pageSize = 500 thì server xử lý thế nào?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, trong mã nguồn EventService.cs, nhóm đã thiết lập trần tối đa (Max Page Size): Math.Min(pageSize, 100). Do đó dù client gửi 500 thì server chỉ lấy tối đa 100 bản ghi, nhằm bảo vệ cơ sở dữ liệu không bị cạn kiệt tài nguyên.

❓ Câu 4: Khi chạy, tệp cơ sở dữ liệu nằm ở thư mục nào?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, dự án VISIONARY sử dụng PostgreSQL trên nền tảng Supabase Cloud chứ không dùng tệp SQLite cục bộ. Chuỗi kết nối được cấu hình thông qua biến môi trường ConnectionStrings:DefaultConnection trỏ tới máy chủ Supabase qua cổng 5432.

V. CHECKLIST TỰ LUYỆN TRƯỚC BUỔI BẢO VỆ

- [ ] Mở sẵn Swagger trên trình duyệt: https://visionary-api-gb51.onrender.com/swagger.
- [ ] Mở sẵn VS Code trỏ đúng vào EventsController.cs và Program.cs.
- [ ] Thực hành thao tác thử nghiệm mã 401 (không gửi Token) và mã 403 (gửi Token Sinh viên vào API Admin) trên Swagger.
- [ ] Luyện tập nói Slide 8, 9, 10, 11 bấm giờ chuẩn dưới 4 phút 15 giây.
- [ ] Nhớ câu kết chuyển giao: 'Sau đây em xin chuyển lời sang bạn Võ Ngọc Minh Đạt...'